

Debugging shapes and gradients

Project: Image-pattern Network Investigation

1. Your mission

Your learning target

I can check dimension contracts before computing a layer.

I can locate the first wrong value in a forward or gradient trace.

I can verify a repair with independent checks and limited claims.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Initialisation and reproducibility. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Training and validation curves. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Training a small network. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Check dimension contracts before computing a layer

Locate the first wrong value in a forward or gradient trace

Verify a repair with independent checks and limited claims

Retrieve what you already know

Why does running the same code twice and getting the same answer not establish correctness?

My starting explanation

This week's end goal

By the end, I can explain debugging shapes and gradients using a traced example and a stated limitation.

2. Notice the evidence

For this lesson, a dense layer stores one output unit per weight-matrix row. If an input has two features and the layer has three units, W has three rows and two columns, b has length 3, and one input x has length 2. Compute z for each unit, then apply ReLU to each component.

Use W rows $[1, 0]$, $[0, 1]$, $[1, -1]$ and $b = [0, 0, 1]$. For $x = [1, 2]$, the scores are $[1, 2, 0]$ and the activations are $[1, 2, 0]$. A batch stores examples in rows: inputs $[[1, 2], [3, 1]]$ have shape 2 by 2, and outputs have shape 2 by 3.

Object · Declared shape · Meaning

Object: One input x

Declared shape: Length 2

Meaning: Two features

Object: Weights W

Declared shape: 3 by 2

Meaning: Three units each reading two features

Object: Bias b

Declared shape: Length 3

Meaning: One bias per unit

Object: Batch output

Declared shape: 2 by 3

Meaning: Two examples each with three activations

Other software may use a transposed weight convention. The key is to record and follow one contract. An

array with the right number of values can still be wrong if features or axes are swapped.

List the given facts and one fact that is not supplied.

3. See how it works

Python zip stops when its shortest input ends. Without a length check, a missing feature can silently remove a contribution. Reject malformed dimensions before computing: do not pad, truncate or reshape a mismatched input just to make the calculation run.

```
def dense_relu(x, weights, bias):  
    if not x or not weights:  
        raise ValueError("Need input features and units")  
    if len(bias) != len(weights):  
        raise ValueError("One bias per unit required")  
    if any(len(row) != len(x) for row in weights):  
        raise ValueError("Each unit needs one weight per feature")  
    scores = [sum(w * value for w, value in zip(row, x)) + b  
              for row, b in zip(weights, bias)]  
    return scores, [max(0.0, z) for z in scores]  
  
weights = [[1, 0], [0, 1], [1, -1]]  
bias = [0, 0, 1]  
for x in [[1, 2], [3, 1]]:  
    print(dense_relu(x, weights, bias))  
# ([1, 2, 0], [1, 2, 0])  
# ([3, 1, 3], [3, 1, 3])
```

The function assumes finite numeric values; it checks dimensions, not every possible input fault. Evaluate the two batch rows independently and stack their outputs. Retain tests that reject a missing input feature, a short weight row and an incorrect bias count.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary

words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

Compute scores and activations for $x = [3, 1]$ with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: The program changes its declared order to $[c, a]$ and returns $[-12, -8]$.

Prediction: That vector now matches the declared coordinate order.

Reason: Gradient values are meaningful only when their parameter names and array positions agree.

5. Words that unlock the idea

Word · Meaning

Word: Shape

Meaning: The sizes of an array along its declared axes

Word: Contract

Meaning: A stated condition that inputs and outputs must satisfy

Word: Partial gradient

Meaning: The change rate of a scalar loss with respect to one parameter while others are fixed

Word: Regression check

Meaning: A saved check rerun after a repair to detect the return of a defect

Use two terms to explain a step in this week's example.

6. Follow a worked example

Repair from the first disagreement

Trace stage for x [1 2] · Expected result · Submitted result

Trace stage for x [1 2]: Input and dimensions

Expected result: x length 2 W 3 by 2 b length 3

Submitted result: Dimensions match

Trace stage for x [1 2]: Scores before ReLU

Expected result: [1, 2, 0]

Submitted result: [1, 2, -1]

Trace stage for x [1 2]: Hidden activations

Expected result: [1, 2, 0]

Submitted result: [1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At $x = [3, 1]$, the same omission changes the third activation from 3 to 2.

Trace $x = [0, 2]$ with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debugging shapes and gradients

Catch a gradient -order bug before the optimiser changes parameters.

The facts we will use

Toy $y=2a+3c$, target 4, start $[a, c]=[1, 0]$. Loss $(y-4)^2$. Program claims gradient $[-12, -8]$ in order $[a, c]$.

All inputs and outcomes in this worked example are invented classroom data; no real model run is claimed.

Debugging shapes and gradients

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Compute correct gradient

$y = 2$, residual -2 : gradient $[-8, -12]$, not $[-12, -8]$.



2. Check coordinates

Step 0.1: a losses 3.24, 4.84 give -8 ; c losses 2.89, 5.29 give -12 .



3. Apply correct update

At rate 0.01: new $[a, c] = [1.08, 0.12]$; $y = 2.52$, loss 2.1904.

CHANGE ONE THING

The program changes its declared order to $[c, a]$ and returns $[-12, -8]$.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Compute correct gradient

What happens and why: $y = 2$, residual -2 : gradient $[-8, -12]$, not $[-12, -8]$.

Step: 2. Check coordinates

What happens and why: Step 0.1: a losses 3.24, 4.84 give -8 ; c losses 2.89, 5.29 give -12 .

Step: 3. Apply correct update

What happens and why: At rate 0.01: new $[a, c] = [1.08, 0.12]$; $y = 2.52$, loss 2.1904.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Array shape and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter. Label the entries a and c , then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

What this example does not establish

A correct gradient at one point does not establish every batch, shape or activation path is implemented correctly.

Read the labelled picture: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.

Change one thing: The program changes its declared order to $[c, a]$ and returns $[-12, -8]$. Predict the result before checking the explanation. What remains the same?

7. Find and repair a mistake

Use a separate scalar diagnostic model $y = 2w + b$, target 3, loss $(y - 3)^2$, and parameter order $[w, b]$. At $[1, 0]$, prediction is 2, residual -1 and gradient $[-4, -2]$. A submitted gradient $[-2, -4]$ has the correct length but swapped meanings.

With symmetric step 0.1, probe w while b stays 0: losses at $w = 0.9$ and 1.1 are 1.44 and 0.64. Their difference divided by 0.2 is -4. Probe b while w stays 1: losses at $b = -0.1$ and 0.1 are 1.21 and 0.81, giving -2. Use original parameters for both probes and compare by coordinate with a stated tolerance.

A gradient check must include the actual loss convention and any penalty. For ReLU, avoid probes crossing its corner at zero; a symmetric value there need not be a derivative. A passing numerical check at selected points is evidence about those points, not a proof for every input.

Locate the earliest unsupported step

For the diagnostic model at $[1, 0]$, gradient $[-4, -2]$ and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.

State one repair and the checks you would rerun.

8. Practise together

The layer expects W shape 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed contract and repair process.

Trace $x = [0, 2]$ with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

Explain your prediction to a partner before comparing results.

Practise with less help: Expose a hidden bias error

Use workbook W2 for the connected example “Debugging shapes and gradients”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case.

Each person writes their own explanation.

Check your reasoning

Use the worked case for Debugging shapes and gradients. Proposed explanation: "Matching output shape guarantees the computation". Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

Compute scores and activations for $x = [3, 1]$ with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

```
def dense_relu(x, weights, bias):
    if not x or not weights:
        raise ValueError("Need input features and units")

    if len(bias) != len(weights):
        raise ValueError("One bias per unit required")

    if any(len(row) != len(x) for row in weights):
        raise ValueError("Each unit needs one weight per feature")

    scores = [sum(w * value for w, value in zip(row, x)) + b
               for row, b in zip(weights, bias)]

    return scores, [max(0.0, z) for z in scores]
```

```

weights = [[1, 0], [0, 1], [1, -1]]
bias = [0, 0, 1]
for x in [[1, 2], [3, 1]]:
    print(dense_relu(x, weights, bias))
# ([1, 2, 0], [1, 2, 0])
# ([3, 1, 3], [3, 1, 3])

```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

The layer expects W shape 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed contract and repair process.

For the diagnostic model at $[1, 0]$, gradient $[-4, -2]$ and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.

A program passes only the $x = [1, 2]$ final-activation test. Propose one intermediate-value check, one malformed-input check and one additional input that exposes the bias omission.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled debugging shapes and gradients evidence sheet to Image-pattern Network Investigation. Show how you can check dimension contracts before computing a layer. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For your Image-pattern Network Investigation, save the input, procedure, observed result and limitation of this check. Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

12. Show what you know

At $[w, b] = [1, 0]$, use the verified gradient $[-4, -2]$ and rate 0.05. Subtracting rate times gradient gives $[1.2, 0.1]$, prediction 2.5 and loss 0.25. A sign bug that adds it gives $[0.8, -0.1]$, prediction 1.5 and loss 2.25. Recompute loss after the complete update instead of trusting an intended direction.

A repair makes one final output match but leaves the third score wrong. Another repair makes loss decrease but has swapped gradient entries. Are either of these sufficient implementation checks? Describe the evidence needed.

Lower loss alone cannot prove a correct gradient or update; some wrong steps also improve loss. Retain shape rejections, hand-computed intermediate traces, numerical coordinate checks and update checks. Next week investigates the representations produced by hidden layers.

A program passes only the $x = [1, 2]$ final-activation test. Propose one intermediate-value check, one malformed-input check and one additional input that exposes the bias omission.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Repair from the first disagreement

Trace stage for x [1 2] · Expected result · Submitted result

Trace stage for x [1 2]: Input and dimensions

Expected result: x length 2 W 3 by 2 b length 3

Submitted result: Dimensions match

Trace stage for x [1 2]: Scores before ReLU

Expected result: [1, 2, 0]

Submitted result: [1, 2, -1]

Trace stage for x [1 2]: Hidden activations

Expected result: [1, 2, 0]

Submitted result: [1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At $x = [3, 1]$, the same omission changes the third activation from 3 to 2.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Debug a new gradient independently

Use $y = 3a + c$, target 2, loss $(y - 2)^2$, at $[a, c] = [0, 1]$. A program returns gradient $[-2, -6]$ in order $[a, c]$. Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Use $y = 3a + c$, target 2, loss $(y - 2)$ squared, at $[a, c] = [0, 1]$. A program returns gradient $[-2, -6]$ in order $[a, c]$. Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.

15. Your lesson route

Start with this question: Catch a gradient -order bug before the optimiser changes parameters.

By the end, I can · How I will show it

By the end, I can: Check dimension contracts before computing a layer

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Locate the first wrong value in a forward or gradient trace

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Verify a repair with independent checks and limited claims

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project evidence. Your teacher may spread this lesson across several classes.

Keep your first prediction. Compare it with what the evidence shows and explain any change.

16. Finish and return

Close your notes. Explain this goal in your own words: Check dimension contracts before computing a layer

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an input or assumption and justify your prediction.

Save your workbook evidence. At the next review, try a different case before opening your old answer.

17. Supported practice, repair and portfolio

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Catch a gradient -order bug before the optimiser changes parameters.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? At rate 0.01: new $[a, c] = [1.08, 0.12]$; $y = 2.52$, loss 2.1904.

1. Compute correct gradient
2. Check coordinates
3. Apply correct update

My selected conclusion, deciding fact and explanation

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

After your attempt, compare with the supported explanation: Apply correct update At rate 0.01: new $[a, c] = [1.08, 0.12]$; $y = 2.52$, loss 2.1904. Array shape and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter. Label the entries a and c , then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

Practice 2: explain the deciding evidence

Changed-case practice: The program changes its declared order to $[c, a]$ and returns $[-12, -8]$. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future input.
2. That vector now matches the declared coordinate order.
3. We can decide the result without examining the changed input or the procedure.

My selected conclusion, deciding fact and explanation

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

After your attempt, compare with the supported explanation: That vector now matches the declared coordinate order. Gradient values are meaningful only when their parameter names and array positions agree.

Repair a missing building block

Review Initialisation and reproducibility (ai-5/17). Goal: Trace how starting values affect a fixed training budget. Short practice: Does the matching final loss mean the parameter values are equal? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Review Training and validation curves (ai-5/15). Goal: Compute comparable training and validation losses at saved checkpoints. Short practice: Can the original epoch 3 training loss 1 justify claiming best generalisation? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Review Training a small network (ai-5/12). Goal: Trace repeated forward loss gradient and update steps. Short practice: Which parameter changed, and which part was not trained? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual output or error, change one input, then run again. Keep code, predictions, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run evidence. Open the HTML copy to read or print it. Drafts stay in this browser when storage is available; clear drafts on a shared device.